

# Notes on installing and using CVMFS for open data analysis

Paulius Kalinauskas, 19<sup>th</sup> of September 2026

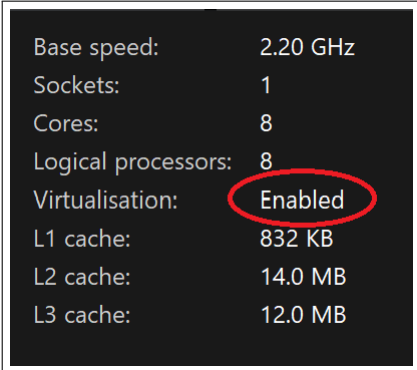
## 0 General remarks

I installed the CVMFS system onto a WSL2 window following the document “Unofficial guide: "Setting Up AlmaLinux 9 for LHCb Development" provided by Dominykas Stonkus (e-mail: dominykas.stonkus@cern.ch); thus, the notes are structured around said document.

This is not a complete guide by any means, but only my general remarks about issues I ran into when installing the system.

## 1 Installation of WSL2

You must have virtualization turned on in your computer for WSL2 to work properly. You can check if it's turned on by going to the Performance tab of your Task Manager:



|                     |          |
|---------------------|----------|
| Base speed:         | 2.20 GHz |
| Sockets:            | 1        |
| Cores:              | 8        |
| Logical processors: | 8        |
| Virtualisation:     | Enabled  |
| L1 cache:           | 832 KB   |
| L2 cache:           | 14.0 MB  |
| L3 cache:           | 12.0 MB  |

Figure 1: Virtualization settings in Task Manager

If virtualization is not turned on, you can do so through the BIOS menu (you can follow any guide on how to do so, for instance [this one](#)).

## 2 Additional dependencies

Following the guide didn't seem to work for me in this section. By running the commands:

```
sudo dnf install \  
epel-release \  
bzip2-devel \  
cppunit-devel \  
...  
-y
```

I kept running into issues because the CodeReady Builder (crb) wasn't installed and enabled beforehand.

```

client-devel xrootd-devel xrootd-server -y
[sudo] password for KalinauskasSummerProject:
Last metadata expiration check: 0:07:09 ago on Mon 27 Jul 2026 04:55:06
PM EEST.
No match for argument: cppunit-devel
No match for argument: fuse-devel
No match for argument: json-c-devel
Package zlib-devel-1.2.11-40.el9.x86_64 is already installed.
No match for argument: xrootd-client-devel
No match for argument: xrootd-devel
No match for argument: xrootd-server
Error: Unable to find a match: cppunit-devel fuse-devel json-c-devel xro
otd-client-devel xrootd-devel xrootd-server

```

Figure 2: Issues with additional dependencies

This is fixed by changing the order of the commands and enabling crb before running everything else:

```
sudo dnf install epel-release -y
```

```
sudo dnf config-manager --set-enabled crb
```

```

sudo dnf install \
bzip2-devel \
cppunit-devel \
curl-devel \
...
-y

```

### 3 Building the local LHCb stack

Here, I ran into an issue when configuring the utils/config.json file, where I omitted the "+g" tag at the end of the following line:

```
"binaryTag": "x86_64_v3-el9-gcc13-opt+g"
```

This caused issues later on, when CMake failed to find the requested Gaudi version. I would advise to simply keep the "+g" tag, as the guide suggests.

The following command:

```
make Davinci
```

Uses a lot of computing power, and shuts down if enough resources are not provided. This can be fixed by opening up a Windows PowerShell terminal (one without the Alma environment open), and running the following command:

```
notepad $env:USERPROFILE\.wslconfig
```

This avoids issues with Windows automatically attaching a .txt extension. Then, give the environment more computing power by adjusting the parameters given in the setup document. For instance:

```
[wsl2]
memory=8GB
processors=2
swap=4GB
```

Note that these changes typically only come into effect after a restart of the PowerShell.

## 4 After a restart

After every restart, CVMFS doesn't remount automatically, so one has to either type in the following commands into PowerShell (the Alma environment specifically):

```
sudo cvmfs_config setup
sudo cvmfs_config wsl2_start
sudo mount --make-shared /cvmfs
cvmfs_config probe source /cvmfs/lhcb.cern.ch/lib/LbEnv
```

Or automate this process using a [boot] command in /etc/wsl.conf. One also has to move to their working directory. In my case, this was /data, so the command would be:

```
cd ~ /data
```

Here, the tilde brings one to their home directory.

## 5 Tips for the analysis of .dst files

I ran into issues with the version of DaVinci I was using. I read that one needed to match the version of DaVinci to the one that was used to create the .dst files. This turns out to be false, and it's best to simply use the latest version. I was using an old version of v39r1p6 and kept running into a lot of issues with my Python files not outputting anything. One should instead use newer versions like v45r8 when running their programs. For instance:

```
lb-run DaVinci/v45r8 python file_name.py
```

Instead of:

```
lb-run DaVinci/v39r1p6 python file_name.py
```

It is a good idea to start with something simple, getting used to the environment. I recommend trying to copy the example given by the [LHCb open data guide](#). It introduces how to create NTuples from the .dst files provided by the open data portal. The rest of the guide can also be read for an introduction on how the data is actually processed.

Due to the sheer amount of data that needs to be processed when creating NTuples, I would highly recommend reading the lesson provided by the [LHCb Starterkit](#) on how to run several jobs in parallel across multiple processors. This saves a great amount of time when working with large data chunks.

If possible, an Ethernet connection should be used when analyzing .dst files directly from the open data portal. Even with multiple processors running at once, the speed of the analysis can be connection-limited if using Wifi.

## 6 Accuracy of the data

If you are working with Run 1 data, don't expect the results to be arbitrarily accurate. [This article](#) from 2014 details a systematic uncertainty of 0.03% on the momentum scale in the LHCb experiments performed from 2010 to 2012, which propagates into all subsequent measurements (page 28). Here they used large samples of  $J/\psi \rightarrow \mu^+\mu^-$  and  $B^+ \rightarrow J/\psi K^+$  decays. Coincidentally, I used the same ones for my [summer project](#) and also obtained mass offsets of the same order of magnitude. Therefore, this data is known to have a global momentum-scale bias, and one shouldn't expect the results to match with the known values completely.

Feel free to take a look at my [Github repository](#) for inspiration, or a starting point for you analysis. I tried to detail every part of the project to the best of my abilities, but I don't claim that my code is optimal, so feel free to play around with it.

## References

- [1] An example of creating NTuples: <https://lhcb-opendata-guide.web.cern.ch/run1-dsts-odp/examples/simple-example/>, last accessed 5<sup>th</sup> of August 2026.
- [2] LHCb Starterkit lesson on running jobs in parallel: <https://lhcb.github.io/starterkit-lessons/first-analysis-steps/split-jobs.html>, last accessed 5<sup>th</sup> of August 2026.
- [3] 2014 paper on LHCb Detector Performance: <https://arxiv.org/pdf/1412.6352>, last accessed 19<sup>th</sup> of September 2026.
- [4] My Github repository: <https://github.com/KalinauskasSummerProject/SummerProjectTryout>, last accessed 19<sup>th</sup> of September 2026.